Variables, bucles y condicionales

Nombre: _____ Fecha: _____

Qué vas a aprender

- Entender qué es una variable y usarla para guardar y modificar datos
- Distinguir tipos de datos: números, texto y booleanos
- Usar bucles con contador y con condición
- Tomar decisiones con condicionales y operadores lógicos

Para pensar

¿Cómo sabe un videojuego cuántos puntos llevas, cuántas vidas te quedan y cuándo has ganado?

Explicación

1.1 ¿Qué es una variable?

Una **variable** es un espacio con nombre en la memoria del ordenador donde se guarda un dato que puede cambiar. Imagina una caja con una etiqueta: la etiqueta es el nombre (`puntos`) y dentro está el valor (`120`).

Con una variable se pueden hacer tres cosas:

- **Crearla** y darle un valor inicial: `puntos ← 0`.
- **Leerla**: usar su valor en un cálculo o mostrarlo.
- **Modificarla**: `puntos ← puntos + 10` (toma el valor actual, le suma `10` y lo guarda).

Elige nombres **claros**: `vidas` es mejor que `v` o `x2`.

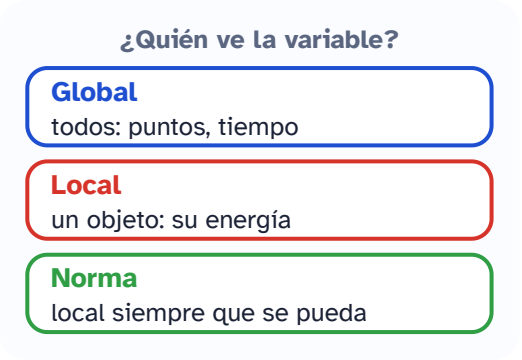


1.2 Variables globales y locales

En Scratch, al crear una variable hay que elegir entre «**para todos los objetos**» y «**solo para este objeto**». Una variable **global** la pueden leer y cambiar todos los objetos y el escenario: la puntuación del juego o el tiempo restante. Una variable **local** pertenece a un solo objeto y cada copia de ese objeto tiene la suya: la energía de cada enemigo o la velocidad de cada coche en una carrera.

Elegir bien evita errores difíciles de encontrar: si la energía de los enemigos fuera global, al golpear a uno bajaría la de todos. En los lenguajes de texto ocurre lo mismo: una variable creada dentro de una función solo existe dentro de ella (es local) y desaparece al terminar. Como norma, conviene usar variables locales siempre que sea posible y reservar las globales para los datos que de verdad

comparte todo el programa. Además, las variables se pueden **mostrar** u **ocultar** en el escenario: mostrar las importantes (puntos, vidas) y ocultar las internas.



1.3 Tipos de datos

No todos los datos son iguales. Los lenguajes distinguen **tipos**:

Tipo	Ejemplos	Operaciones
Número entero	0, 15, –3	+, –, ·, /, comparar
Número decimal	3,14 · 9,8	igual, con decimales
Texto (cadena)	"Hola", "Ana"	unir, medir longitud
Booleano	verdadero / falso	y, o, no

Cuidado: `"5" + "5"` como texto da `"55"` (se unen), pero `5 + 5` como números da `10`. Una **lista** (o *array*) guarda muchos valores con un solo nombre: `notas = [7, 5, 9]`.



1.4 Listas: muchos datos en una variable

Una **lista** (o array) guarda muchos valores bajo un mismo nombre, ordenados por su **posición** o índice. En Scratch, la lista «compra» puede contener «pan»,

«leche» y «huevos»: el elemento 1 es «pan» y el 3, «huevos». Con los bloques de listas se puede añadir un elemento al final, insertar en una posición, borrar, reemplazar, consultar la **longitud** (cuántos elementos hay) y saber si la lista **contiene** un valor.

Las listas se recorren con un bucle: para i desde 1 hasta longitud(lista), hacer algo con el elemento i. Así se puede sumar todas las notas de una lista, buscar la más alta o contar cuántos aprobados hay. Sirven para guardar las preguntas de un concurso, el récord de diez jugadores o los puntos de un recorrido. En Python se escriben entre corchetes, **compra = ["pan", "leche", "huevos"]**, y la primera posición es la 0, no la 1.



1.5 Contadores y acumuladores

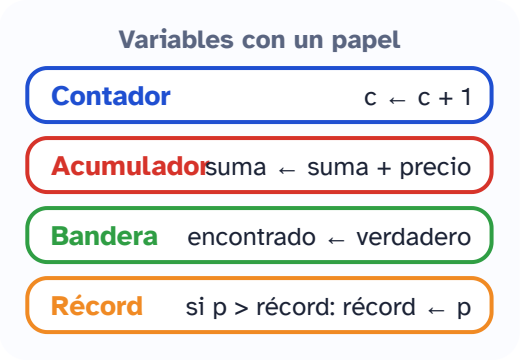
Dos usos muy frecuentes de las variables:

- **Contador**: aumenta de uno en uno para contar sucesos. `intentos ← intentos + 1`. Sirve para contar vueltas de un bucle, saltos o enemigos eliminados.
- **Acumulador**: va sumando cantidades variables. `total ← total + precio`. Sirve para calcular el total de una compra o la suma de las notas.

Ambos deben inicializarse (normalmente a 0) antes de empezar. Si no, arrastran el valor de la partida anterior: un error típico en Scratch es olvidar «dar a puntos el valor 0» al pulsar la bandera verde.

Existen otros usos típicos. Un **interruptor** o bandera es una variable booleana que indica si algo ha ocurrido: encontrado ← falso antes de buscar y encontrado ← verdadero cuando aparece. Una variable de **máximo** o de **récord** guarda el mayor valor visto: si puntos > récord, entonces récord ← puntos. Todas necesitan un

valor inicial adecuado: 0 para contadores y acumuladores, falso para las banderas y un valor muy pequeño (o el primer dato) para el máximo.



2.1 Bucles con contador

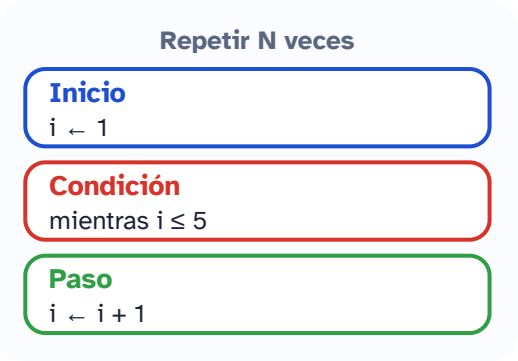
El bucle **repetir N veces** (o **para**, `for` en inglés) se usa cuando sabemos cuántas repeticiones necesitamos. Suele llevar una variable contadora que cambia en cada vuelta:

```
para i desde 1 hasta 5
    mostrar i · i
fin para
```

Muestra 1, 4, 9, 16 y 25: los cuadrados del 1 al 5. Cada vuelta del bucle se llama **iteración**.

Los bucles evitan copiar el mismo código muchas veces. Para dibujar un polígono de 12 lados no escribimos 24 bloques, sino «repetir 12: mover 30, girar 30°».

El contador no tiene por qué avanzar de uno en uno: «para i desde 0 hasta 100 de 10 en 10» recorre 0, 10, 20… 100, y «para i desde 10 hasta 1 con paso −1» cuenta hacia atrás, útil para una cuenta atrás. El número de vueltas de «para i desde a hasta b» (de uno en uno) es b − a + 1: desde 3 hasta 8 son 6 vueltas. La variable contadora se puede usar dentro del bucle, por ejemplo para mostrar la tabla de multiplicar: mostrar 7 · i.



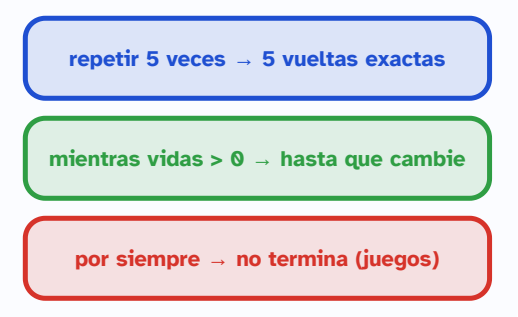
2.2 Bucles con condición

Cuando no sabemos cuántas veces hay que repetir, usamos un bucle con **condición**:

- **Mientras** (`while`): repite mientras la condición sea verdadera. Si es falsa desde el principio, no se ejecuta nunca.
- **Repetir hasta que** (en Scratch): repite hasta que la condición se cumpla.
- **Por siempre**: repite sin fin; típico de los juegos, que revisan continuamente lo que pasa.

```
vidas ← 3
repetir hasta que vidas = 0
    jugar ronda
    si pierdo entonces vidas ← vidas − 1
mostrar "Game over"
```

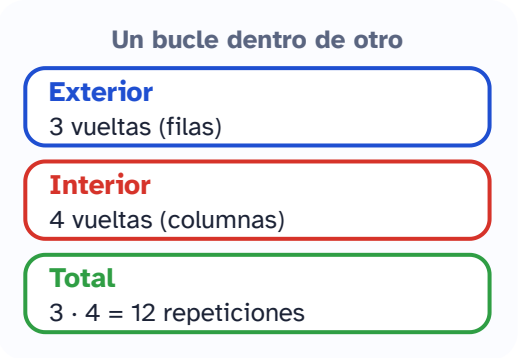
Para no crear un **bucle infinito** sin querer, dentro del bucle debe haber siempre alguna instrucción que pueda hacer que la condición cambie: sumar al contador, leer un dato nuevo o mover el objeto. Si el programa se queda congelado, lo primero es revisar los bucles «mientras» y «repetir hasta que» y comprobar qué variable debería cambiar en cada vuelta.



2.3 Bucles anidados

Un bucle puede estar dentro de otro: son los **bucles anidados**. Por cada vuelta del bucle exterior, el interior se ejecuta completo. Si el exterior da 3 vueltas y el interior 4, las instrucciones de dentro se ejecutan $3 \cdot 4 = 12$ veces. Es la estructura que se usa para recorrer filas y columnas, como en una tabla de multiplicar completa, un tablero de ajedrez o los píxeles de una imagen.

Ejemplo: para dibujar una cuadrícula de 5 × 5 cuadrados, el bucle exterior recorre las filas y el interior dibuja los 5 cuadrados de cada fila; al terminar una fila, se baja y se vuelve al principio. Otro ejemplo: para i de 1 a 10, para j de 1 a 10, mostrar i · j, escribe las cien multiplicaciones de las tablas del 1 al 10. Hay que usar **variables distintas** en cada bucle (i y j) y vigilar el total de repeticiones, porque crece muy deprisa: tres bucles de 100 anidados son un millón de vueltas.



3.1 Condicionales

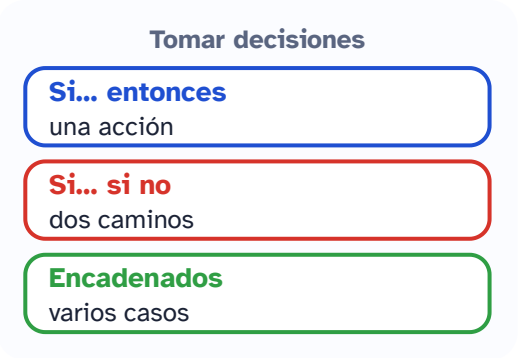
Un **condicional** permite al programa tomar decisiones. Evalúa una condición (que es verdadera o falsa) y ejecuta un bloque u otro:

- **si… entonces** (`if`): solo actúa si se cumple.
- **si… entonces… si no** (`if… else`): dos caminos.
- **Condicionales anidados** o encadenados (`elif`): varias opciones.

```
si nota ≥ 9 entonces mostrar "sobresaliente"
si no, si nota ≥ 7 entonces mostrar "notable"
si no, si nota ≥ 5 entonces mostrar "aprobado"
si no mostrar "suspenso"
```

El orden importa: se comprueba de arriba abajo y se ejecuta solo la primera opción verdadera.

En los condicionales encadenados el **orden** de las condiciones importa, porque se ejecuta solo la primera que se cumple. Para clasificar notas hay que empezar por la más exigente (≥ 9) o por la más baja (< 5) y avanzar en orden. Y no hay que confundir varios «si» independientes, que se comprueban todos, con un «si… si no si…», en el que solo se ejecuta uno.

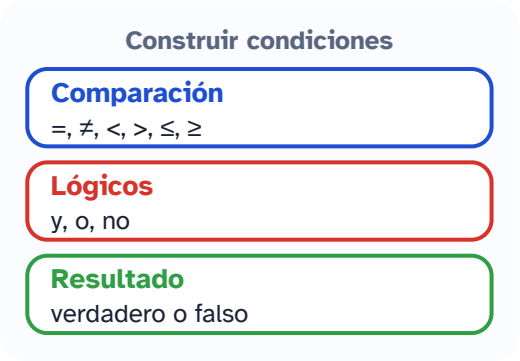


3.2 Operadores de comparación y lógicos

Las condiciones se construyen con **operadores**:

Operador	Significado	Ejemplo verdadero
= · ≠	igual · distinto	3 = 3 · 4 ≠ 5
< · >	menor · mayor	2 < 7
≤ · ≥	menor o igual · mayor o igual	5 ≥ 5
y (and)	verdadero si se cumplen las dos	edad ≥ 12 y edad ≤ 16
o (or)	verdadero si se cumple al menos una	día = sábado o día = domingo
no (not)	invierte el valor	no (llueve)

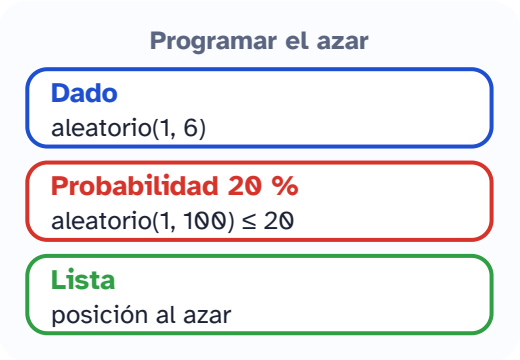
Los resultados de estas operaciones son **booleanos**: verdadero o falso, en honor al matemático George Boole.



3.3 El azar: números aleatorios

Muchos programas necesitan **azar**: un dado, la posición en la que aparece un enemigo, el orden de las preguntas de un concurso o una carta al azar. El bloque «número aleatorio entre 1 y 6» de Scratch (random.randint(1, 6) en Python) devuelve un número distinto cada vez. Se combina con variables y condicionales: dado ← aleatorio(1, 6); si dado = 6, mostrar «¡Repites turno!».

El azar también sirve para **probabilidades**: si queremos que algo ocurra el 20 % de las veces, generamos un número entre 1 y 100 y comprobamos si es menor o igual que 20. Con listas, se puede elegir un elemento al azar usando como posición un número aleatorio entre 1 y la longitud. En realidad, los ordenadores generan números **pseudoaleatorios**: los calculan con una fórmula a partir de una «semilla», como la hora exacta, pero parecen aleatorios y bastan para juegos y simulaciones.



4.1 Todo junto: un ejemplo completo

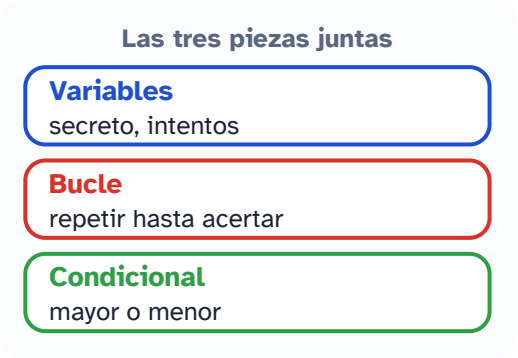
Un juego de adivinar un número combina variables, bucle y condicionales:

```
secreto ← número al azar entre 1 y 100
intentos ← 0
repetir
```

```
    preguntar "¿Qué número es?"
    intentos ← intentos + 1
    si respuesta < secreto entonces decir "Mayor"
    si respuesta > secreto entonces decir "Menor"
hasta que respuesta = secreto
decir "¡Acertaste en " + intentos + " intentos!"
```

Identifica cada pieza: dos **variables** (`secreto` e `intentos`), un **contador**, un **bucle** con condición y dos **condicionales**.

Fíjate en el papel de cada pieza: **secreto** e **intentos** son variables; intentos es un contador; el bucle «repetir hasta que» se ejecuta mientras no se acierte; y el condicional decide qué pista dar. Para mejorarlo se puede limitar el número de intentos (condición compuesta: hasta que acierte **o** intentos = 7) y guardar el récord de la partida con menos intentos.

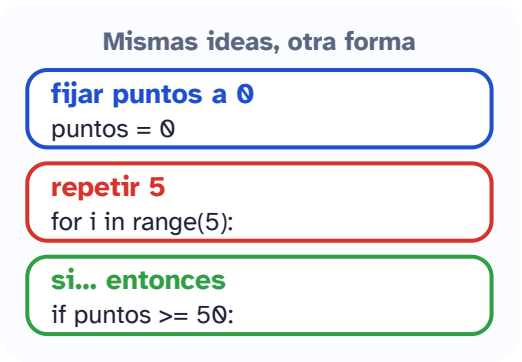


4.2 Del bloque al texto

Los mismos conceptos se escriben de forma muy parecida en los lenguajes de texto. En **Python**, uno de los más usados para aprender:

```
puntos = 0
for i in range(5):
    puntos = puntos + 10
if puntos >= 50:
    print("¡Nivel superado!")
```

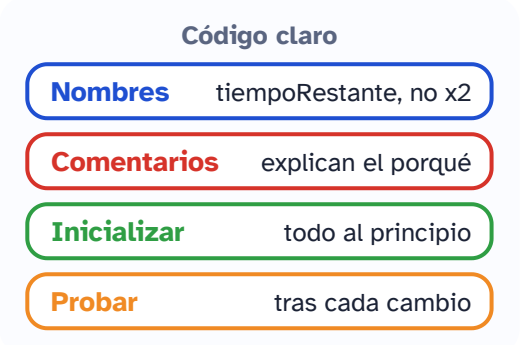
Fíjate en que Python usa la **sangría** (los espacios al principio de la línea) para saber qué instrucciones están dentro del bucle o del condicional, igual que los bloques de Scratch se encajan unos dentro de otros. Aprender la lógica primero hace que el paso a un lenguaje escrito sea mucho más sencillo.



4.3 Buenas prácticas: nombres claros y comentarios

Un programa se lee muchas más veces de las que se escribe, por eso debe ser fácil de entender. La primera regla es usar **nombres de variables descriptivos**: «vidas», «tiempoRestante» o «precioTotal» dicen mucho más que «a», «x2» o «cosa». No deben llevar espacios (se usa camelCase, como tiempoRestante, o guiones bajos, como tiempo_restante) y conviene no empezarlos por un número ni usar tildes en los lenguajes de texto.

La segunda regla es **comentar**: en Scratch se pueden añadir notas amarillas a los scripts, y en Python todo lo que va detrás de # es un comentario que el ordenador ignora. Un buen comentario explica **por qué** se hace algo, no lo que ya es evidente. Otras buenas prácticas: inicializar todas las variables al principio, agrupar los scripts por objetos y funciones, evitar copiar y pegar el mismo código (usar bloques propios) y probar el programa a menudo, después de cada pequeño cambio, en lugar de escribirlo todo de golpe.



Las piezas de cualquier programa

Variable	nombre + valor que cambia · inicializar
Tipos	número · texto · booleano · lista
Contador	c ← c + 1 · acumulador: t ← t + x
Bucles	repetir N · mientras · hasta que · por siempre
Condicionales	si · si no · y / o / no

Ejercicios

1. **¿Qué valor final tiene x? x ← 4; x ← x · 3; x ← x – 2.**

2. **Clasifica por tipo: 17, «hola», verdadero, 2,5, «17».**

3. **Escribe un bucle que muestre los números pares del 2 al 20.**

4. **¿Cuántas veces se ejecuta: c ← 10; mientras c > 0: c ← c – 2?**

5. **Escribe un acumulador que sume los precios de 5 productos.**

6. **Evalúa: (7 > 3) y (2 = 5).**

7. **Evalúa: (7 > 3) o (2 = 5).**

8. **Escribe un condicional que clasifique la temperatura: < 15 «frío», 15-25 «agradable», > 25 «calor».**

9. **Corrige: puntos ← 0; repetir 3: puntos + 5; mostrar puntos (muestra 0).**

10. **Programa en Scratch un cronómetro de cuenta atrás de 10 segundos que diga «¡Tiempo!» al terminar.**

11. **¿Qué diferencia hay entre una variable global y una local en Scratch?**

12. **Una lista contiene «rojo», «verde», «azul». ¿Qué elemento está en la posición 2? ¿Cuál es su longitud?**

13. **¿Qué bloque usarías para simular un dado de 20 caras?**

14. **Propón nombres claros para variables que guarden: las vidas, el tiempo que falta y el nombre del jugador.**

15. **¿Para qué sirve una variable bandera? Pon un ejemplo.**

16. **¿Qué hace el símbolo # en Python?**

17. **¿Cuántas veces se ejecuta el interior de dos bucles anidados de 5 y 8 vueltas?**

18. **Escribe el algoritmo que busca el número mayor de una lista.**

19. **Programa una pregunta de concurso elegida al azar de una lista de 10 preguntas.**

20. **¿Por qué la energía de los enemigos debe ser una variable local?**

21. **Escribe un programa que haga llover: en cada vuelta crea una gota en una x aleatoria entre –240 y 240.**

22. **Añade un récord al juego de adivinar: guarda la partida con menos intentos.**

23. **¿Qué muestra este programa? lista ← [3, 7, 2]; total ← 0; para cada x en lista: si x > 2: total ← total + x.**

24. **Escribe en Python una lista con tus tres asignaturas favoritas y muestra la primera.**

25. **Explica qué son los números pseudoaleatorios.**

26. **Dibuja con bucles anidados un tablero de 8 × 8 casillas alternando colores. Describe el algoritmo.**

27. **Diseña un juego de memoria: el ordenador genera una secuencia aleatoria de colores que crece en cada ronda y el jugador debe repetirla. ¿Qué variables y listas usas?**

28. **Simula 1.000 lanzamientos de un dado y cuenta cuántas veces sale cada número. ¿Qué esperas obtener?**

29. **Encuentra los tres errores del juego de la lectura (puntos que no se reinician, suma mal colocada, meteorito que cuenta dos veces) y escribe la solución.**

30. **Reescribe con buenas prácticas: a ← 0; para b de 1 a 30: a ← a + c[b]; d ← a / 30.**

Practica

1. **Una variable es...**

   A) un bloque de movimiento
   B) un espacio con nombre que guarda un dato que puede cambiar
   C) un error
   D) un sonido

2. **Tras «puntos ← 5; puntos ← puntos + 10», puntos vale...**

   A) 10
   B) 5
   C) 15
   D) 0

3. **Un dato que solo puede ser verdadero o falso es...**

   A) booleano
   B) texto
   C) decimal
   D) lista

4. **«5» + «5» como texto da...**

   A) 10
   B) error
   C) «55»
   D) 5

5. **Una variable que aumenta de 1 en 1 para contar es un...**

   A) acumulador
   B) contador
   C) booleano
   D) bandera

6. **Si sabes cuántas veces repetir, usas...**

   A) repetir N veces (para)
   B) mientras
   C) si... entonces
   D) al azar

7. **Un bucle «mientras» con condición falsa desde el principio...**

A) se ejecuta una vez
B) no se ejecuta nunca
C) es infinito
D) dos veces

8. **«edad ≥ 12 y edad ≤ 16» es verdadero si edad = ...**

A) 10
B) 18
C) 14
D) 12

9. **«día = sábado o día = domingo» es verdadero el...**

A) domingo
B) lunes
C) miércoles
D) lunes y martes

10. **Cada vuelta de un bucle se llama...**

A) variable
B) iteración
C) condición
D) evento

11. **Una variable que pueden usar todos los objetos es...**

A) local
B) global
C) booleana
D) una lista

12. **Una lista guarda...**

A) un solo valor
B) muchos valores ordenados
C) solo números
D) solo texto

13. **En Python, la primera posición de una lista es la...**

A) 1
B) 0
C) –1

D) 10

14. **Dos bucles anidados de 4 y 5 vueltas ejecutan el interior...**

A) 9 veces

B) 20 veces

C) 4 veces

D) 5 veces

15. **Para que algo ocurra el 25 % de las veces se comprueba...**

A) aleatorio(1, 100) ≤ 25

B) aleatorio(1, 4) = 5

C) aleatorio(1, 100) > 25

D) 25 = 25

16. **Una variable que indica si algo ha pasado es una...**

A) bandera

B) lista

C) constante

D) función

17. **El mejor nombre para una variable es...**

A) x

B) a2

C) tiempoRestante

D) cosa

18. **En Python, un comentario empieza por...**

A) //

B) #

C) --

D) *

19. **La longitud de una lista es...**

A) su primer elemento

B) el número de elementos

C) el último valor

D) su nombre

20. **Los ordenadores generan números...**

A) totalmente aleatorios

B) pseudoaleatorios

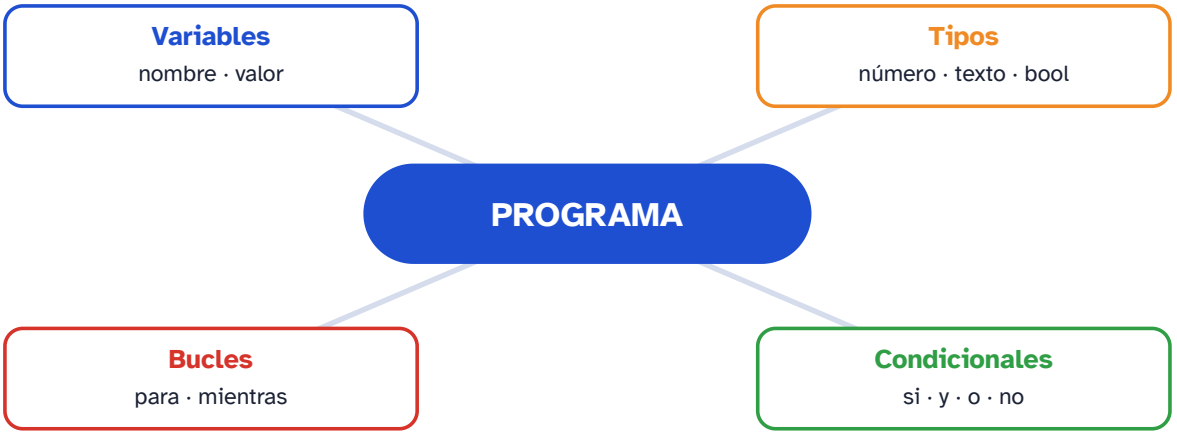
C) siempre iguales

D) solo pares

Explícalo con tus palabras

Incluye: Qué es una variable y cómo se modifica; Diferencia entre contador y acumulador; Cuándo usar repetir N veces y cuándo mientras; Cómo funcionan los operadores y, o, no.

Resumen



Glosario

Variable	Espacio con nombre que guarda un dato que puede cambiar.
Inicializar	Dar valor inicial a una variable.
Variable global	Accesible desde todo el programa.
Variable local	Solo existe para un objeto o función.
Tipo de dato	Clase de valor: número, texto, booleano.
Booleano	Valor verdadero o falso.
Lista	Colección ordenada de valores.
Índice	Posición de un elemento en una lista.
Contador	Variable que aumenta de uno en uno.
Acumulador	Variable que va sumando valores.
Bandera	Variable booleana que indica si algo ocurrió.
Iteración	Cada vuelta de un bucle.
Bucle anidado	Bucle dentro de otro.
Condicional	Estructura que decide según una condición.
Operador lógico	y, o, no.
Número aleatorio	Valor al azar en un intervalo.
Comentario	Nota en el código que no se ejecuta.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Variable

Espacio con nombre que guarda un dato que puede cambiar.

Inicializar

Dar a una variable su valor de partida.

Booleano

Dato que solo vale verdadero o falso.

Contador

Variable que suma 1 cada vez: c ← c + 1.

Acumulador

Variable que va sumando cantidades: t ← t + x.

Iteración

Cada vuelta de un bucle.

Mientras

Bucle que repite mientras la condición sea verdadera.

Operador y

Verdadero solo si las dos condiciones lo son.

Variable global

La comparten todos los objetos.

Lista

Muchos valores bajo un nombre, por posiciones.

Bucle anidado

Un bucle dentro de otro: vueltas multiplicadas.

Número aleatorio

Valor al azar dentro de un intervalo.

Soluciones

1. **B)** Caja con etiqueta.
2. **C)** 5 + 10.
3. **A)** Por George Boole.
4. **C)** Se concatenan.
5. **B)** c ← c + 1.
6. **A)** Bucle con contador.
7. **B)** Comprueba antes de entrar.
8. **C)** Cumple ambas.
9. **A)** Basta una.
10. **B)** Iterar = repetir.
11. **B)** Para todos los objetos.
12. **B)** Por posiciones.
13. **B)** Se empieza a contar en 0.
14. **B)** 4 · 5.
15. **A)** Una de cada cuatro.
16. **A)** Booleana: verdadero o falso.
17. **C)** Descriptivo.
18. **B)** Almohadilla.
19. **B)** Cuántos hay.
20. **B)** Con una fórmula y una semilla.

Soluciones de los ejercicios

1. 4 · 3 = 12; 12 − 2 = 10.
2. Entero, texto, booleano, decimal, texto.
3. para i desde 1 hasta 10: mostrar 2 · i (o para i de 2 a 20 de 2 en 2).
4. 5 veces (10, 8, 6, 4, 2).
5. total ← 0; repetir 5 veces: leer precio; total ← total + precio; mostrar total.
6. Verdadero y falso → falso.
7. Verdadero o falso → verdadero.
8. si t < 15 frío; si no, si t ≤ 25 agradable; si no calor.
9. Debe ser puntos ← puntos + 5; así muestra 15.
10. tiempo ← 10; repetir hasta que tiempo = 0: esperar 1 s; tiempo ← tiempo − 1; decir «¡Tiempo!».
11. La global la usan todos los objetos (puntuación); la local pertenece a un solo objeto y cada copia tiene la suya (energía de cada enemigo).
12. «verde»; longitud 3.
13. número aleatorio entre 1 y 20.
14. vidas, tiempoRestante, nombreJugador.
15. Para indicar si algo ha ocurrido (verdadero o falso). Ejemplo: llaveCogida ← falso al empezar y verdadero al recoger la llave; la puerta solo se abre si llaveCogida es verdadero.
16. Indica un comentario: el texto que va detrás no se ejecuta, sirve para explicar el código.
17. 5 · 8 = 40 veces.
18. mayor ← lista[1]; para i de 2 a longitud: si lista[i] > mayor: mayor ← lista[i]. Al final, mostrar mayor.
19. n ← aleatorio(1, 10); preguntar elemento n de «preguntas»; si respuesta = elemento n de «respuestas»: puntos ← puntos + 1.
20. Porque cada enemigo tiene la suya: si fuera global, al golpear a uno bajaría la energía de todos a la vez.
21. Por siempre: crear clon de gota. Al comenzar como clon: ir a x: aleatorio(−240, 240) y: 180; repetir hasta que y < −170: cambiar y en −5; borrar este clon.
22. Al empezar: si récord = 0, nada. Al acertar: si récord = 0 o intentos < récord: récord ← intentos; mostrar «¡Nuevo récord!».
23. Solo suma los mayores que 2: 3 + 7 = 10.
24. asignaturas = ["Tecnología", "Música", "Matemáticas"]; print(asignaturas[0]) # la primera posición es 0
25. Números calculados con una fórmula a partir de una semilla (como la hora) que parecen aleatorios aunque no lo son del todo; bastan para juegos y simulaciones.
26. Para fila de 1 a 8: para col de 1 a 8: si (fila + col) mod 2 = 0: color blanco; si no: color negro; dibujar casilla; avanzar. Al terminar la fila: volver al inicio y bajar una casilla.
27. Lista secuencia (colores al azar), lista respuesta (lo que pulsa el jugador), variable ronda y bandera error. Cada ronda: añadir un color aleatorio a secuencia, mostrarla, borrar

respuesta, recoger pulsaciones y comparar elemento a elemento; si alguno no coincide, error ← verdadero y fin.

28. Lista contador con 6 ceros. Repetir 1.000: d ← aleatorio(1, 6); contador[d] ← contador[d] + 1. Cada número debería salir unas 167 veces (1.000/6), con pequeñas diferencias por el azar.
29. 1) Al pulsar la bandera: fijar puntos a 0. 2) Poner «sumar 10 a puntos» dentro del bucle que detecta el choque. 3) Usar una variable local «destruido» en cada meteorito: si toca un disparo y destruido = falso: destruido ← verdadero, sumar puntos y esconder.
30. # Media de las notas de la clase sumaNotas ← 0 para alumno de 1 a 30: sumaNotas ← sumaNotas + notas[alumno] mediaClase ← sumaNotas / 30